PIZZA SALES

OVERVIEW

This dataset represents a database for a pizza sales system, designed to analyze customer ordering behavior, product performance, and overall business revenue. It consists of four interconnected tables: orders, order_details, pizzas, and pizza_types.

⇒ The orders table stores information about each unique order ID, date, and time of purchase. This helps in analyzing order trends over time, such as peak sales hours and daily demand patterns.

⇒ The order_details table stores the individual orders to specific pizzas and records the quantity of each item purchased. This table acts as a bridge between orders and products, enabling detailed sales analysis.

⇒ The pizzas table contains information about individual pizza variants, including their size (Small, Medium, Large), price, and associated pizza type. This allows for comparisons across different sizes and pricing strategies.

⇒ The pizza_types table provides descriptive details about each pizza category, such as its name (e.g., Margherita, Pepperoni) and category (Vegetarian or Non-Vegetarian). This table is useful for category-level analysis and understanding customer preferences.

SCHEMA

The dataset follows a relational schema consisting of four tables: orders, order_details, pizzas, and pizza_types. These tables are connected through primary and foreign key relationships to ensure data integrity.

1. Orders → Order_Details (One-to-Many)

The orders table is linked to the order_details table through the order_id field.

- Primary Key (orders): order_id
- Foreign Key (order_details): order_id

Each record in the orders table represents a single transaction, while the order_details table can contain multiple records for the same order. This represents a one-to-many relationship, where one order can include multiple pizza items.

2. Pizzas → Order_Details (One-to-Many)

The pizzas table is connected to the order_details table using the pizza_id field.

- Primary Key (pizzas): pizza_id
- Foreign Key (order_details): pizza_id

Each pizza can appear in multiple orders, and each entry in order_details refers to one specific pizza. This forms another one-to-many relationship.

3. Pizza_Types → Pizzas (One-to-Many)

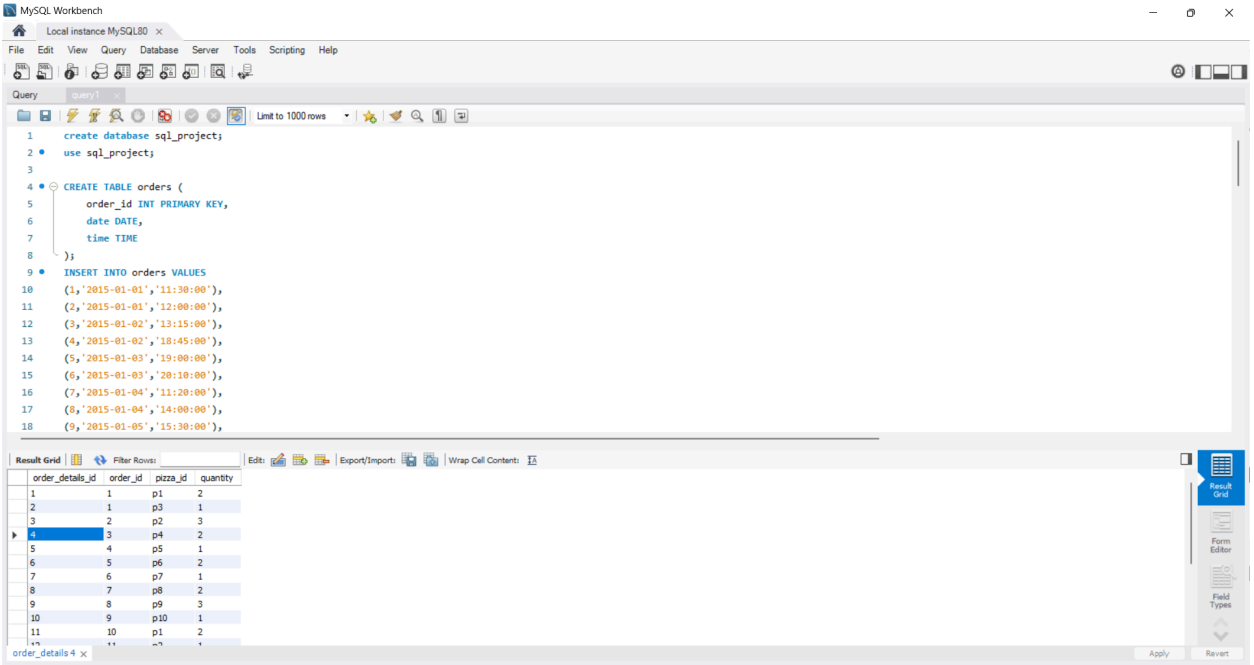
The pizza_types table is linked to the pizzas table via the pizza_type_id field.

- Primary Key (pizza_types): pizza_type_id
- Foreign Key (pizzas): pizza_type_id

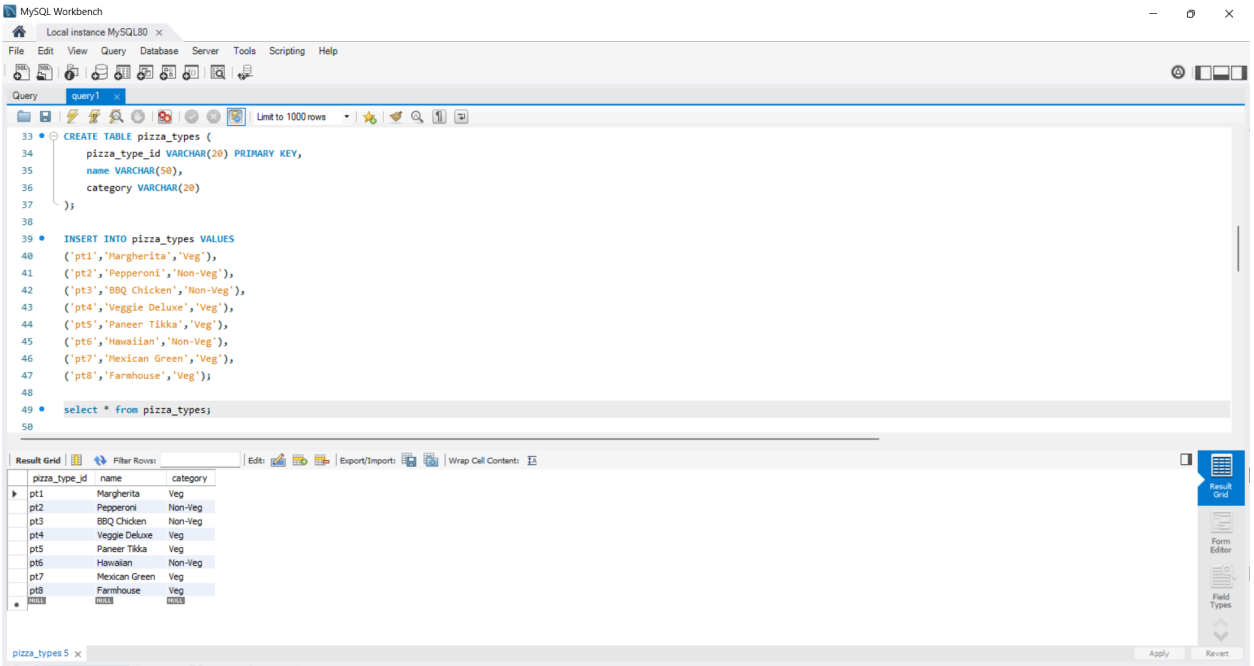
Each pizza type (e.g., Margherita, Pepperoni) can have multiple variations based on size or price. Therefore, this is a one-to-many relationship, where one pizza type maps to multiple pizza records.

CREATING TABLES & INSERTING VALUES

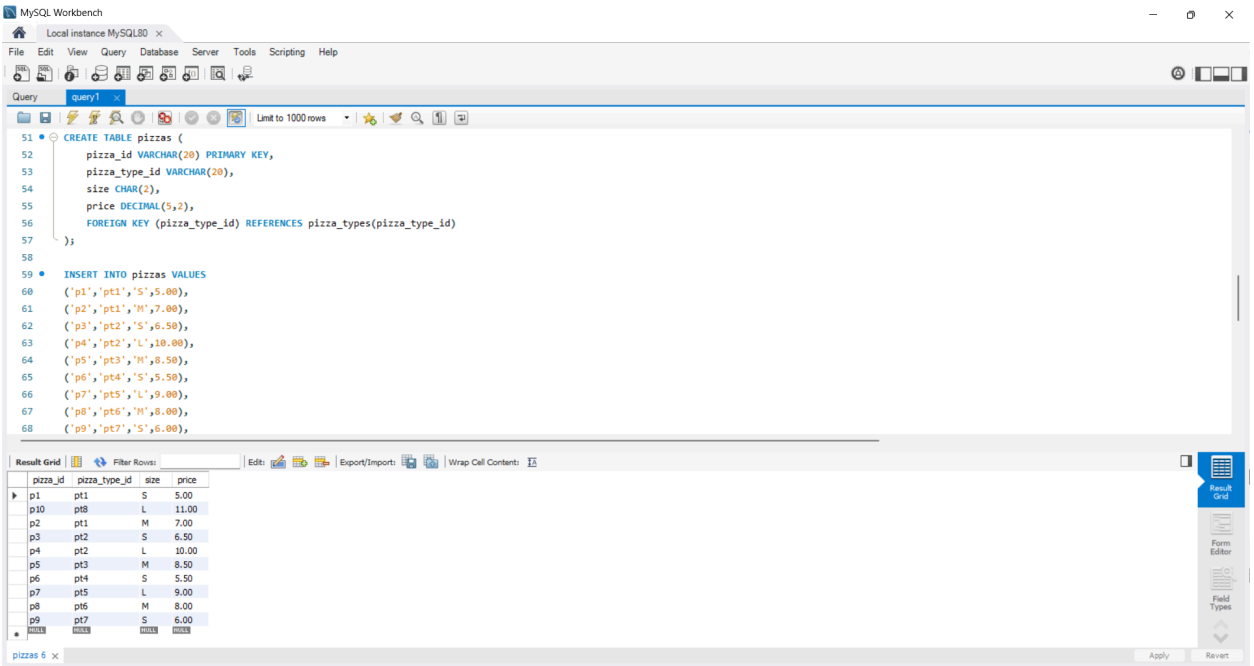
⟶ Orders table



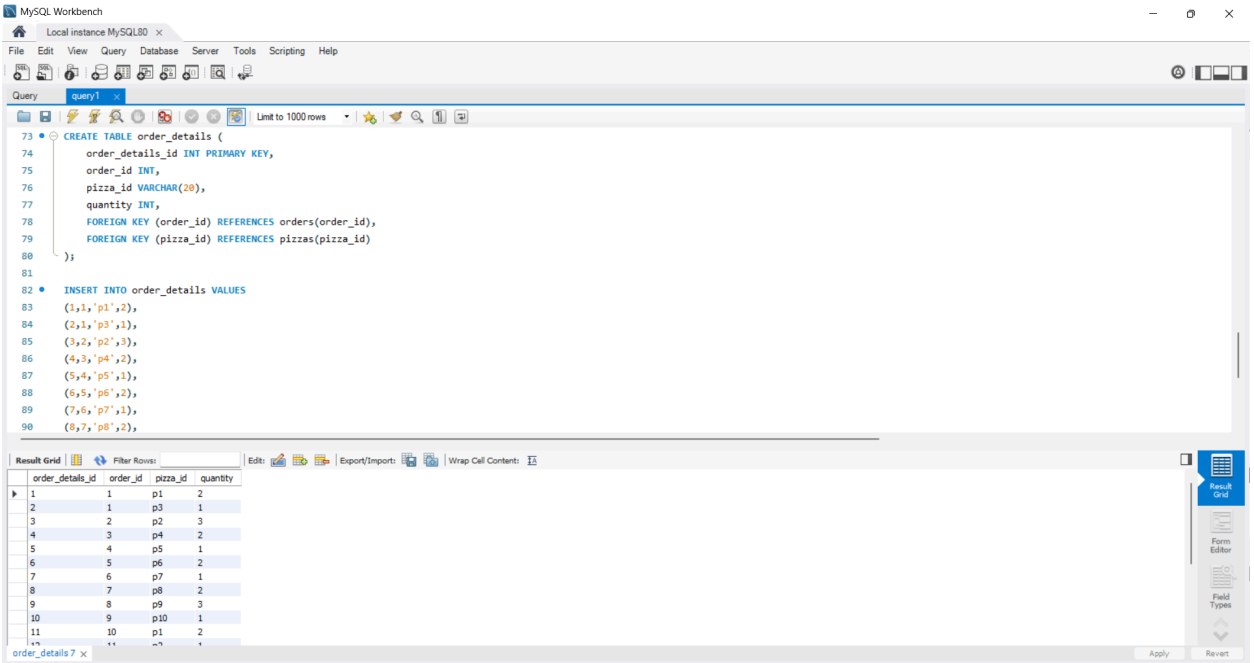
⟶ Pizza_types



→ pizzas



→ Order_details



QUERIES

1. Retrieve the total number of orders placed.

```
-- 1. Retrieve the total number of orders placed.
select
distinct(count(order_id)) as "no.of orders"
from orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	no.of orders
▶	20

2. Get Orders after '2015-01-05'

```sql
-- 2. Get Orders after '2015-01-05'
select
o.order_id,pt.name
from orders as o
join order_details as od
    on o.order_id=od.order_id
join pizzas as p
    on od.pizza_id=p.pizza_id
join pizza_types as pt
    on pt.pizza_type_id=p.pizza_type_id
where o.date > '2015-01-05';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	name
11	Margherita
12	Pepperoni
13	Pepperoni
14	BBQ Chicken
15	Veggie Deluxe
16	Paneer Tikka
17	Hawaiian
18	Mexican Green
19	Farmhouse

3. Calculate the total revenue generated from pizza sales.

```sql
-- 3. Calculate the total revenue generated from pizza sales.
select
sum(p.price*od.quantity) as total_revenue
from pizzas as p inner join order_details as od
    on p.pizza_id=od.pizza_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_revenue
260.50

4. Identify the highest-priced pizza.

```
-- 4. Identify the highest-priced pizza.
select
pt.name, p.price
from pizzas as p
join pizza_types as pt
    on p.pizza_type_id=pt.pizza_type_id
order by p.price
desc limit 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conten

name	price
Farmhouse	11.00

5. Identify the most common pizza size ordered.

```
-- 5. Identify the most common pizza size ordered.
select
p.size,sum(od.quantity) as total_orders
from order_details as od
join pizzas as p
    on od.pizza_id=p.pizza_id
group by p.size
order by total_orders
desc limit 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch

size	total_orders
S	15

6. List the top 5 most ordered pizza types along with their quantities.

```sql
-- 6. List the top 5 most ordered pizza types along with their quantities.
select
p.pizza_type_id, sum(od.quantity) as total_quantity
from order_details od join pizzas p
    on od.pizza_id=p.pizza_id
group by p.pizza_type_id
order by total_quantity
desc limit 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

pizza_type_id	total_quantity
pt1	8
pt2	6
pt3	4
pt4	4
pt6	4

7. List pizzas with sales_qty > 2

```sql
-- 7. List pizzas with sales_qty > 2
select
pizza_id, sum(quantity) as total_qty
from order_details
group by pizza_id
having total_qty>2
order by total_qty desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

pizza_id	total_qty
p1	4
p2	4
p5	4
p6	4
p8	4
p9	4
p10	3
p3	3
p4	3

8. Top 10 Rank pizzas by revenue.

```sql
-- 8. Top 10 Rank pizzas by revenue
select
pizza_id, sum(quantity) as total_qty,
dense_rank() over (order by sum(quantity) desc) as Rank_num
from order_details
group by pizza_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

pizza_id	total_qty	Rank_num
p1	4	1
p2	4	1
p5	4	1
p6	4	1
p8	4	1
p9	4	1
p3	3	2
p4	3	2
p10	3	2
p7	2	3

9. Running total of Revenue.

```sql
-- 9. Running total of Revenue
SELECT
o.date,
SUM(p.price * od.quantity) AS daily_revenue,
SUM(SUM(p.price * od.quantity)) OVER (ORDER BY o.date) AS running_total
FROM orders o
JOIN order_details od ON o.order_id = od.order_id
JOIN pizzas p ON od.pizza_id = p.pizza_id
GROUP BY o.date;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

date	daily_revenue	running_total
2015-01-01	37.50	37.50
2015-01-02	28.50	66.00
2015-01-03	20.00	86.00
2015-01-04	34.00	120.00
2015-01-05	21.00	141.00
2015-01-06	20.00	161.00
2015-01-07	35.50	196.50
2015-01-08	20.00	216.50
2015-01-09	22.00	238.50
2015-01-10	22.00	260.50

10. Categorize pizza price.

```sql
-- 10. Categorize pizza price
select
pizza_id,
price,
case
    when price<10 then 'Low'
    when price between 10 and 20 then 'Medium'
    else 'Expensive'
end as category
from pizzas;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conten

pizza_id	price	category
p1	5.00	Low
p10	11.00	Medium
p2	7.00	Low
p3	6.50	Low
p4	10.00	Medium
p5	8.50	Low
p6	5.50	Low
p7	9.00	Low
p8	8.00	Low
p9	6.00	Low

11. Extract first letter of pizza name

```sql
-- 11. Extract first letter of pizza name
select
left(name,1) as first_letter
from pizza_types;
```

Result Grid | Filter Rows: | Export: | Wrap Cell C

first_letter
M
P
B
V
P
H
M
F

12. Top 5 pizzas by revenue using CTE.

```sql
-- 12. Top 5 pizzas by revenue using CTE
with revenue_cte as (
    select
        p.pizza_id, sum(p.price * od.quantity) as revenue
    from pizzas p
    join order_details as od
        on p.pizza_id=od.pizza_id
    group by p.pizza_id
)
select *
from revenue_cte
order by revenue
desc limit 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

pizza_id	revenue
p5	34.00
p10	33.00
p8	32.00
p4	30.00
p2	28.00

13. Create a view to store total quantity for each pizza type.

```sql
-- 13. create a view to store total quantity for each pizza types
create view pizza_view as
select pt.name,
       SUM(od.quantity) as total_quantity
from order_details od
join pizzas p on od.pizza_id = p.pizza_id
join pizza_types pt on p.pizza_type_id = pt.pizza_type_id
group by pt.name;
```

14. Group the orders by date and calculate the average number of pizzas ordered per day.

```sql
-- 14. Group the orders by date and calculate the average number of pizzas ordered per day.
with daily_orders as (
    select o.date, sum(od.quantity) as daily_total
    from orders as o
    join order_details od
        on o.order_id=od.order_id
    group by o.date
)
select avg(daily_total)
from daily_orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

avg(daily_total)
3.5000

15. Total orders per hour.

```sql
-- 15. Total orders per hour
select
hour(time) as hour,
count(*) as total_orders
from orders
group by hour
order by total_orders desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Co

hour	total_orders
11	3
12	3
13	2
18	2
19	2
20	2
14	2
15	1

Result 32

16. Top 3 busiest hours.

```
-- 16. Top 3 busiest hours
select hour(time) as hour,
count(*) as total_orders
from orders
group by hour
order by total_orders
desc limit 3;
```

Result Grid | Filter Rows: | Export: | W

hour	total_orders
11	3
12	3
13	2

17. Find pizzas with above average prices.

```
-- 17. Find pizzas with above average price
select
pizza_id, price
from pizzas
where price > (select avg(price) from pizzas);
```

Result Grid | Filter Rows: | Edit: | Export/Import:

pizza_id	price
p10	11.00
p4	10.00
p5	8.50
p7	9.00
p8	8.00

18. Find the second highest price pizza.

```
-- 18. Find second highest price pizza
select
max(price) as second_highest
from pizzas
where price < (select max(price) from pizzas);
```

Result Grid | Filter Rows: | Export: | Wrap Cell C

second_highest
10.00

19. Find pizzas that were ordered at least once.

```
-- 19. Find pizzas that were ordered atleast once
select
pizza_id
from pizzas
where pizza_id in (select distinct pizza_id from order_details);
```

Result Grid | Filter Rows: | Edit: | Export/Import:

pizza_id
p1
p2
p3
p4
p5
p6
p7
p8
p9

20. Find the pizza type that generated the highest total revenue.

```sql
-- 20. Find the pizza type that generated the highest total revenue
SELECT
    pt.name AS pizza_name,
    SUM(p.price * od.quantity) AS total_revenue
FROM order_details od
JOIN pizzas p
    ON od.pizza_id = p.pizza_id
JOIN pizza_types pt
    ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.name
ORDER BY total_revenue DESC
LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

pizza_name	total_revenue
Pepperoni	49.50

SUMMARY

The primary objective of this project is to apply core and advanced SQL concepts such as joins, aggregations, subqueries, grouping, filtering, and data modeling to explore sales performance and customer behavior. By leveraging these techniques, the project gives key insights including total revenue generation, most popular pizza types, top-selling sizes, order trends over time, and average daily sales.

The relational structure of the dataset enables efficient querying and ensures data integrity through the use of primary and foreign keys. The order_details table serves as a bridge connecting orders and products, allowing for detailed transactional analysis.

Overall, this project demonstrates strong proficiency in SQL for data analysis and showcases the ability to transform raw transactional data into actionable insights.